Conduit Control Center Product Roadmap

Includes: R1 Capability Audit · D1 UX/UI Design Milestone

Document	
Version	
Date	
Status	
Author	

Table of Contents

Document: CCC_Product_Roadmap_v1

Revision: 1.1

Date: 2026-06-10

Status: Draft for Review

Author: CCC Development Team

1. Overview

Conduit Control Center (CCC) is an open-source web dashboard for managing Psiphon Conduit nodes on Linux servers and Raspberry Pi devices. Its goal is to give volunteer operators a secure, lightweight, user-friendly interface that eliminates the need for direct CLI access for day-to-day operations.

This roadmap documents the planned evolution of CCC from a monitoring dashboard (v0.1) to a full control centre capable of configuring Conduit, visualising live and historical traffic, intelligently assisting operators with resource decisions, and managing personal relay identities.

Target platform: Raspberry Pi 4 (4 GB), Ubuntu 22.04 ARM64, public IP, Cloudflare DNS, Nginx, FastAPI, Python 3.

2. Guiding Principles

- **Open-source first.** No proprietary dependencies. No telemetry without consent.
- **Security by default.** Never store or expose private keys, pairing tokens, or API secrets. Principle of least privilege throughout.
- **Beginner-friendly installation.** `install.sh` should work for a first-time Linux user.
- **Lightweight.** Suitable for Raspberry Pi 4 (4 GB). Avoid heavy frameworks or background indexing.
- **Production-ready.** Systemd integration, Nginx TLS, proper logging, graceful restarts.
- **Capability-driven.** No feature is scoped until it is confirmed possible with the installed Conduit binary.

3. R1 — Capability Audit (Research Milestone)

> **Gate:** No v0.2.0 implementation begins until R1 is complete and the capability matrix is approved.

3.1 Audit Scope

Five areas were audited against the official Psiphon-Inc/conduit repository (commit history current as of 2026-06-10):

- **A.** Personal Mode (Personal Links, Pairing URLs, QR generation, identities, invitations)
- **B.** Client Limits (max common, max personal, scopes, runtime configurability)
- **C.** Bandwidth Controls (global, per-direction, reduced/scheduling, runtime configurability)
- **D.** Metrics Inventory (complete `conduit_*` Prometheus gauge catalogue)
- **E.** Ryve (claim workflow, QR, rewards metadata, mobile integration)

Sources examined: `cli/cmd/start.go`, `cli/cmd/ryve.go`, `cli/internal/config/config.go`, `cli/internal/config/compartment.go`, `cli/internal/metrics/metrics.go`, `cli/README.md`, `cli/GUIDE.md`.

3.2 Conduit Linux CLI Capability Matrix

The table below captures every capability assessed. "CCC Candidate" describes the proposed CCC surface for that capability.

#	Capability	Android App	Linux CLI	CCC Current	CCC Candidate
A — Personal Mode					
A1	Enable personal clients	✓	✓ `--max-personal-clients`	✗	Read + Write (v0.4.0)
A2	Compartment ID generation	✓	✓ `new-compartment-id` cmd	✗	Generate + persist (v0.4.0)
A3	Personal pairing token (v1)	✓	✓ `BuildPersonalPairingToken`	✗	Generate + display (v0.4.0)
A4	Personal pairing QR code	✓	✓ (token → QR)	✗	Display in UI (v0.4.0)
A5	`--compartment-id` accepts raw ID or token	—	✓ `NormalizePersonalCompartmentInput`	✗	Accepts both formats (v0.4.0)
A6	Pairing name max length	32 chars	✓ 32 chars enforced	✗	Validated input (v0.4.0)
A7	Compartment persisted to disk	—	✓ `personal_compartment.json`	✗	Read persisted ID (v0.4.0)
A8	Per-user invitation management	✓	✗ Not in CLI	✗	Out of scope
B — Client Limits					
B1	Max common clients (0–1000)	✓	✓ `--max-common-clients` (default 50)	✗ read	Read + Write (v0.2.0)
B2	Max personal clients (0–1000)	✓	✓ `--max-personal-clients` (default 0)	✗	Read (v0.2.0), Write (v0.4.0)
B3	At-least-one-client validation	—	✓ error if both = 0	—	Enforce in UI (v0.2.0)
B4	Runtime client limit update	—	✓ `SetConfig(common, personal, bw)`	✗	Apply via service restart (v0.2.0)
B5	`InproxyMaxCommonClients` via `--set`	—	✓ in allowlist	—	Covered by B1

#	Capability	Android App	Linux CLI	CCC Current	CCC Candidate
B6	`InproxyMaxPersonalClients` via `--set`	—	✗ not in `--set` allowlist	—	Use `--max-personal-clients` flag (v0.4.0)
C — Bandwidth Controls					
C1	Global bandwidth limit (Mbps)	✓	✓ `--bandwidth` (default 40, -1 unlimited)	✗ read	Read + Write (v0.2.0)
C2	Bandwidth stored as bytes/sec	—	✓ `int(Mbps × 1,000,000 / 8)`	—	Display as Mbps in UI (v0.2.0)
C3	Separate upstream/downstream limits	✓	✓ via JSON config only (`InproxyLimit*BytesPerSecond`)	✗	Out of scope for v0.2.0; consider v0.3.0
C4	Unlimited bandwidth (`-1`)	✓	✓	✗	Toggle in UI (v0.2.0)
C5	Runtime bandwidth update	—	✓ `SetConfig(…, bandwidthBytesPerSecond)`	✗	Apply via service restart (v0.2.0)
C6	Reduced-mode bandwidth scheduling	—	✓ `InproxyReducedStartTime / EndTime / Limit*` via `--set`	✗	Scheduling UI (v0.2.0)
C7	Reduced common clients during schedule	—	✓ `InproxyReducedMaxCommonClients` via `--set`	✗	Scheduling UI (v0.2.0)
C8	Minimum throttle protection	—	✓ 100 GB / 7 days enforced by `conduit-monitor`	—	Document in UI tooltip (v0.2.0)
D — Metrics					
D1	`conduit_announcing`	—	✓	✓	Already exposed
D2	`conduit_connecting_clients`	—	✓	✓	Already exposed
D3	`conduit_connected_clients`	—	✓	✓	Already exposed
D4	`conduit_is_live` (broker connection)	—	✓	✗	Add to Live Ops (v0.2.0)
D5	`conduit_max_common_clients`	—	✓	✓	Already exposed

#	Capability	Android App	Linux CLI	CCC Current	CCC Candidate
D6	`conduit_max_personal_clients`	—	✓	✗	Show in Config panel (v0.2.0)
D7	`conduit_bandwidth_limit_bytes_per_second`	—	✓ (0 = unlimited)	✗	Show in Config panel (v0.2.0)
D8	`conduit_bytes_uploaded`	—	✓ cumulative	✗	Show in Live Ops (v0.2.0)
D9	`conduit_bytes_downloaded`	—	✓ cumulative	✗	Show in Live Ops (v0.2.0)
D10	`conduit_uptime_seconds`	—	✓ GaugeFunc (computed at scrape)	✗	Show in Live Ops (v0.2.0)
D11	`conduit_idle_seconds`	—	✓ GaugeFunc (computed at scrape)	✗	Show in Live Ops (v0.2.0)
D12	`conduit_region_bytes_uploaded{scope,region}`	—	✓	✗	Regional Analytics (v0.2.0)
D13	`conduit_region_bytes_downloaded{scope,region}`	—	✓	✗	Regional Analytics (v0.2.0)
D14	`conduit_region_connecting_clients{scope,region}`	—	✓	✗	Regional Analytics (v0.2.0)
D15	`conduit_region_connected_clients{scope,region}`	—	✓	✗	Regional Analytics (v0.2.0)
D16	`conduit_build_info{build_repo,build_rev,go_version,values_rev}`	—	✓	partial	Expose full build info (v0.2.0)
D17	Scope labels: `common` / `personal`	—	✓	✗	Scope filter in Regional Analytics (v0.2.0)
E — Ryve					
E1	`conduit ryve-claim` command	—	✓	✗	Subprocess invocation (v0.4.0)

#	Capability	Android App	Linux CLI	CCC Current	CCC Candidate
E2	Claim URI (`network.ryve.app://…`)	—	✓	✗	Display claim QR (v0.4.0)
E3	Claim QR PNG saved to `data/ryve-claim-qr.png`	—	✓	✗	Serve PNG from data dir (v0.4.0)
E4	ProxyID (Curve25519 public key)	—	✓	✗	Display ProxyID (v0.4.0)
E5	Private key exposure in claim	—	✓ requires explicit confirmation	✗	**CCC must NEVER store or display private key**
E6	Ryve rewards data	—	✗ not in CLI source	✗	Out of scope until confirmed
E7	Station name / identity label	—	✓ `--name` flag on `ryve-claim`	✗	Pre-fill with hostname (v0.4.0)

3.3 Confirmed Findings Summary

Personal Mode is fully supported in Linux CLI via `--max-personal-clients` and `--compartment-id`. A compartment ID must be generated first; the CLI errors if personal > 0 and no compartment ID is configured. Personal pairing tokens use a defined v1 format (base64url-encoded JSON `{v:"1", data:{id, name}}`). CCC v0.4.0 can implement the full personal-mode workflow.

Client limits are runtime-configurable via `SetConfig()`, which updates common clients, personal clients, and bandwidth in a single call. Since CCC manages Conduit via systemd, "apply" in practice means writing new flags to the service unit drop-in and restarting — not an in-process hot-reload.

Bandwidth defaults to 40 Mbps (decimal, stored as bytes/sec). The `--bandwidth` flag controls a combined limit; separate upstream/downstream limits require JSON config overrides and are not exposed on the `--set` allowlist. Reduced-mode scheduling (`InproxyReduced*` via `--set`) is the correct mechanism for time-based throttling, including the v0.2.0 bandwidth scheduling feature.

`conduit_is_live` is a confirmed metric (1 = broker connected, 0 = disconnected) and is not yet surfaced in CCC. It should be the topmost status signal on the dashboard in v0.2.0.

Ryve is a mobile app that claims a Conduit station by scanning a QR code encoding the station's private keypair. CCC must never store or display the private key itself. The safe workflow is: invoke `conduit ryve-claim --output <path>` as a subprocess (after explicit user confirmation), serve the resulting QR PNG temporarily, then delete it. The private key must never be logged, stored in the database, or transmitted over the API.

`InproxyMaxPersonalClients` **is not in the** `--set` **allowlist.** Personal client limits must be set via the `--max-personal-clients` flag, not `--set`. The Config panel write path in v0.4.0 must use `--max-personal-clients N` as a service unit flag — not `--set InproxyMaxPersonalClients=N`, which silently does nothing.

4. v0.1.1 — Maintenance Release

v0.1.1 is a maintenance-only release. No new user-facing features. All items correspond to open GitHub issues.

Issue	Title	Priority
#3	Review and correct the unsupported `conduit pair`-based pairing workflow in `adapter.pair()` / `/api/conduit/pair`. `conduit pair` is not a CLI subcommand in any Conduit release (confirmed on `main` and `release-cli-2.0.0`). Full pairing functionality is planned for v0.4.0 (Section 8).	High
#4	Restrict `ccc.db` file permissions (644 → 600)	High
#5	Remove duplicate DDNS cron entry from root crontab	Medium
#6	Update UFW firewall rules in `pre-install.md` based on production validation	Low

Release criteria: all four issues resolved, tests green, CHANGELOG updated.

5. D1 — UX/UI Design Milestone (Gate for v0.2.0)

> **Gate:** No v0.2.0 implementation begins until the D1 design deliverables are approved. The dashboard must be designed by, or to the standard of, a professional frontend/UX designer before a single line of implementation code is written.

The dashboard must not be a raw metrics dump. Every screen must be something a non-technical volunteer operator — someone who set up a Raspberry Pi for the first time — can understand and act on without reading documentation.

5.1 Design Principles

Principle	Requirement
User-friendly	Operators must understand every value on screen without knowing Prometheus, systemd, or Go.
WYSIWYG where possible	Configuration changes must preview their effect before being applied.
Non-technical clarity	No raw metric names (`conduit_bytes_uploaded`), no internal identifiers, no backend jargon exposed to the user.
Clean and modern	Visual design consistent with contemporary web dashboards (e.g. Grafana, Vercel, Linear).
Mobile-friendly	Fully usable on a smartphone. Operators must be able to check station health and change limits from their phone.
Lightweight	No heavy JavaScript frameworks. Must load acceptably on a Raspberry Pi 4 over a slow mobile connection.

5.2 Information Architecture

The dashboard is organised into eight named sections. These sections drive both the navigation structure and the grouping of all features across v0.2.0–v0.4.0.

#	Section	Contents
1	**Overview**	Station status, health indicator, key metrics summary — the home screen
2	**Live Clients**	Connected + connecting clients, broker status, uptime, idle time
3	**Traffic**	Total upload/ download, bandwidth limit display
4	**Regions**	Top active countries with flags, client counts, traffic volumes
5	**Configuration**	Max common clients, max personal clients, bandwidth limit, apply workflow
6	**Scheduling**	Bandwidth schedule profiles (redu ced-mode)
7	**Smart Assistant**	Manual / Assisted / Automatic mode selector and analysis panel
8	**Personal & Ryve**	Personal mode setup, pairing tokens, Ryve claim QR (v0.4.0)

Settings (theme, password, session) are accessible via a gear icon, separate from the operational sections above.

5.3 First-Screen Requirements

The Overview (home screen) is the first thing an operator sees after login. It must answer the operator's three questions in under three seconds:

1. **Is my station working?** → Station status badge (Live / Starting / Offline / Disconnected)
2. **Is anyone using it?** → Connected clients count (large, prominent)
3. **How much have I served?** → Total upload + download since last restart

The first screen must contain, above the fold on a standard desktop browser:

Element	Value shown	Notes
Station status badge	Live / Starting / Offline / Disconnected	Colour-coded: green / yellow / grey / red
Health indicator	Good / Warning / Critical	Derived from status + idle time + client count
Connected clients	Integer	Large metric card
Connecting clients	Integer	Secondary metric card
Bandwidth limit	X Mbps or Unlimited	Secondary metric card
Upload total	Human-readable (KiB/MiB/GiB/TiB)	Secondary metric card
Download total	Human-readable (KiB/MiB/GiB/TiB)	Secondary metric card
Top active countries	Top 3–5 country names + client counts	Compact summary; links to full Regions section
Uptime	Xd Xh Xm	Secondary detail

Nothing on the first screen should require the operator to already know what they are looking for.

5.4 Navigation Model

Desktop: persistent left sidebar with section icons and labels. Active section highlighted. Sidebar collapses to icon-only at narrower widths. No horizontal top navigation.

Mobile: bottom navigation bar with icons for the five most important sections (Overview, Live Clients, Traffic, Regions, Configuration). Remaining sections accessible via a "More" item. No hamburger menu.

Navigation rules:

- The current section title is always visible in the page header.
- Breadcrumbs are not required — sections are flat, not nested.
- Destructive or disruptive actions (apply config, restart service) are never reachable in one tap/click from the navigation. They require entering the Configuration section first.
- Deep-linking to sections is supported (`/dashboard#regions`, `/dashboard#config`).

5.5 Visual Hierarchy Rules

These rules apply to every screen in the dashboard:

Rule	Detail
Large metric cards	Used for the 3–4 most critical values on a given section (e.g. connected clients, station status). Font size ≥ 32px for the number.
Secondary cards	Used for supporting values (e.g. connecting clients, uptime). Smaller but same card component.
Tables	Only used when a list of comparable rows is genuinely the clearest format (e.g. Regions table). Never used for single values or key metrics.
Charts	Only added when the shape of data over time improves a decision (v0.3.0 onwards). Not in v0.2.0 except where explicitly called out.
No raw metric names	Never display `conduit_bytes_uploaded`, `conduit_is_live`, or any Prometheus/internal identifier to the user. Use plain English labels.
No raw ISO codes alone	Country codes (e.g. `MM`, `IR`) must always be accompanied by the country name and flag emoji.
Colour usage	Green = good/live/healthy. Yellow/amber = warning/transitioning. Red = error/offline. Grey = unknown/stopped. Blue = informational/neutral. No other status colours.
Empty states	Every section that can have zero data must display a friendly empty-state message explaining what the data will show once available (e.g. "No clients connected yet. New stations can take a few hours to receive traffic.").

5.6 Human-Readable Values

All values presented to the operator must use human-readable formats. No raw numbers without units; no internal representations.

Value type	Display format	Example
Byte counts (transfer totals)	Binary prefixes: KiB, MiB, GiB, TiB	`14.3 GiB`
Bandwidth limit	Decimal Mbps (matches CLI) or "Unlimited"	`40 Mbps` / `Unlimited`
Duration (uptime, idle)	`Xd Xh Xm Xs` — omit leading zero units	`2d 4h` / `37m 12s`
Client counts	Plain integer	`127`
Country identifiers	Flag emoji + full country name + ISO code in parentheses	Myanmar (MM)
Conduit version	Semantic version string from build info	`v2.0.0`
Timestamps	Local time in `YYYY-MM-DD HH:mm`	`2026-06-10 14:23`
Bandwidth in/out rates (v0.3.0)	Human-readable binary + `/s`	`1.2 MiB/s`

Never mix decimal and binary byte prefixes on the same panel. Mbps (decimal) is used exclusively for the configured bandwidth limit value, as this matches the CLI's `--bandwidth` flag. All other byte quantities use binary prefixes (KiB, MiB, GiB, TiB).

5.7 Settings UX

The Configuration section manages all parameters that affect Conduit's behaviour. Because changes here cause a service restart, every interaction in this section must be deliberate and

reversible.

Layout: a single form with clearly labelled fields, grouped into logical sub-sections (Client Limits, Bandwidth, Scheduling). A persistent "Apply Changes" button is anchored to the bottom of the form. The button is disabled until the operator has actually changed at least one value from the current state.

Field-level UX:

Field	Input type	Constraints shown
Max common clients	Number input + slider	Range: 0–1000. Live validation. Warning if set to 0 while personal clients also 0.
Max personal clients	Read-only display in v0.2.0	"Personal mode not yet configured" placeholder.
Global bandwidth limit	Number input (Mbps) + "Unlimited" toggle	Minimum: 1 Mbps. Displays estimated impact (e.g. "~X clients at average usage").
Smart Assistant mode	Three-way toggle: Manual / Assisted / Automatic	Automatic mode requires a separate confirmation dialog before enabling.

Save/apply workflow:

1. Operator changes one or more values. Changed fields are visually highlighted (yellow left border).

2. A diff summary appears below the form: "You are changing: Max common clients 50 → 75".

3. Operator clicks "Apply Changes".

4. A confirmation modal appears with the diff and the warning: "Applying these changes will briefly restart Conduit. Connected clients will be disconnected for a few seconds."

5. Operator confirms. CCC writes the drop-in, restarts the service, and shows a progress indicator.

6. On success: a green toast notification. Fields reset to the new current values.

7. On failure: a red toast with the error. No changes are retained in the running service.

Bandwidth schedule UX:

- Displayed as a visual timeline (24h bar) with the scheduled reduced-bandwidth window highlighted.

- Fields: start time, end time, reduced bandwidth (Mbps), reduced max clients (optional).
- Schedule type selector: Weekdays / Weekends / Every day / Custom (day checkboxes).
- A tooltip explains: "During this window, Conduit will use reduced settings. Values below 100 GB / 7 days are overridden by the Conduit traffic monitor."
- Changes to the schedule follow the same diff + confirm + restart workflow as other configuration changes.

5.8 Warnings, Confirmations, and Errors

The dashboard uses three levels of notification:

Level	Trigger	Presentation
Toast (non-blocking)	Success actions, non-critical state changes	Bottom-right corner, auto-dismiss after 4s. Green (success) or red (error).
Banner (persistent)	Station offline, broker disconnected, health degraded	Top of the relevant section, stays until condition resolves. Yellow or red. Dismissible.
Modal (blocking)	Any action that restarts Conduit, enables Automatic mode, or invokes Ryve claim	Requires explicit confirmation click. Shows exactly what will change and what the consequence is. Cancel button always present.

Specific warning text requirements:

- **Before applying configuration changes:** "Applying these changes will briefly restart Conduit. Clients currently connected will be disconnected for a few seconds. Do you want to continue?"
- **Before enabling Automatic mode:** "Automatic mode will adjust your client limits without asking for confirmation. You can set minimum and maximum bounds below. Are you sure you want to enable Automatic mode?"
- **Before Ryve claim (v0.4.0):** "Generating the Ryve claim QR code requires accessing your station's private key. The key will not be stored or transmitted — only the QR code image will be displayed. This should only be done in a private location. Continue?"
- **Station offline banner:** "Your Conduit station is not running. Check the service status or review the logs."
- **Broker disconnected banner:** "Conduit is running but not connected to the Psiphon broker. No clients can be served until the connection is restored."

- **New station empty state:** "No clients connected yet. New stations can take several hours to receive traffic while building reputation with the Psiphon network."

5.9 Mobile Behaviour

The dashboard must be fully functional on a 375px-wide smartphone screen.

Component	Mobile behaviour
Sidebar navigation	Hidden; replaced by bottom navigation bar (5 primary sections + More)
Metric cards	Stack vertically, full width
Configuration form	Stacked single-column layout; sliders replaced by number inputs with +/− buttons
Regions table	Simplified: show flag, country name, clients only. Traffic columns hidden (accessible via row expand)
Modals	Full-screen on small viewports
Charts (v0.3.0)	Horizontally scrollable within their container
Toast notifications	Bottom-centre, full width

Touch targets: all interactive elements (buttons, inputs, toggles, navigation items) must have a minimum touch target of 44×44px.

Performance: the dashboard must be usable on a 3G mobile connection. Total initial page payload (HTML + CSS + JS, excluding metrics API calls) must not exceed 300 KB uncompressed.

5.10 Theme Support

Three theme modes: **Light**, **Dark**, and **System** (follows OS preference).

Implementation requirements:

- Implemented using CSS custom properties (`--color-bg`, `--color-surface`, `--color-text`, etc.) — no JavaScript required to switch themes.
- Theme preference persisted via a server-side cookie (`theme=light|dark|system`) set on the settings endpoint — not `localStorage` (not available in the embedded environment).
- System theme uses `@media (prefers-color-scheme: dark)` as the CSS fallback.
- Theme toggle is accessible from all screens via a small icon in the header or sidebar footer.
- All status colours (green/yellow/red/grey) must meet WCAG AA contrast in both light and dark themes.
- Charts and tables must have theme-aware colour palettes.

5.11 Pre-Implementation Deliverables

Before any v0.2.0 implementation code is written, the following design artefacts must be produced and approved:

Deliverable	Description
Dashboard layout proposal	Annotated wireframe or mockup showing the layout of each section on desktop and mobile. Must demonstrate the navigation model, card hierarchy, and form layout for Configuration.
Section hierarchy document	Written description of each section's content, the information priority within it, and how it relates to adjacent sections.
Card/table structure specification	For each metric card and table in the dashboard: the data source, the display format, the empty state, and the error state.

Deliverable	Description
Navigation model diagram	Diagram showing how a user moves between sections, what is reachable from the Overview, and what requires deliberate navigation to reach.
Mobile wireframe	Wireframe of the bottom navigation bar and at least three sections in mobile layout.
UX rule document	Written enumeration of all warning messages, confirmation dialogs, and error states specified in Section 5.8, plus any additional cases identified during design.

6. v0.2.0 — Smart Conduit Control

> **Gate:** D1 design deliverables must be approved before implementation begins.

v0.2.0 transforms CCC from a read-only monitoring dashboard into an interactive control centre. The central theme is: **display everything Conduit exposes and give operators control over the parameters they need most.**

6.1 Conduit Configuration

Display and allow editing of the three runtime-configurable parameters:

- **Max common clients** — current value from `conduit_max_common_clients`; editable (0–1000); enforces at-least-one-client rule.
- **Max personal clients** — current value from `conduit_max_personal_clients`; read-only in v0.2.0 (write deferred to v0.4.0 pending personal-mode setup flow).
- **Global bandwidth limit** — current value from `conduit_bandwidth_limit_bytes_per_second`; display as Mbps (0 = unlimited); editable with unlimited toggle.

Apply mechanism: write new flags to the systemd service unit drop-in (`/etc/systemd/system/conduit.service.d/ccc.conf`) and run `systemctl daemon-reload && systemctl restart conduit`. Confirm to the user that Conduit will restart briefly (see Section 5.8).

> **Note:** `conduit_bandwidth_limit_bytes_per_second` uses decimal megabits (1 Mbps = 125,000 bytes/sec). Display and input must use decimal Mbps to match CLI behaviour.

6.2 Live Operations Panel

Live Operations section, aligned with the card structure defined in Sections 5.3 and 5.5:

Metric	Source	Display Format
Broker status	`conduit_is_live`	Live / Starting / Offline / Disconnected badge
Connected clients	`conduit_connected_clients`	Large metric card
Connecting clients	`conduit_connecting_clients`	Secondary metric card
Uptime	`conduit_uptime_seconds`	`Xd Xh Xm`

Metric	Source	Display Format
Idle time	`conduit_idle_seconds`	`Xd Xh Xm` or "Active"
Bytes uploaded	`conduit_bytes_uploaded`	Human-readable binary (KiB/MiB/GiB/TiB)
Bytes downloaded	`conduit_bytes_downloaded`	Human-readable binary (KiB/MiB/GiB/TiB)
Build revision	`conduit_build_info`	`v2.0.0 · short-rev`

Four-state broker status logic:

- `conduit_announcing == 0` and service not reachable → **Not running** (grey)
- `conduit_announcing == 1` and `conduit_is_live == 0` → **Starting** (yellow)
- `conduit_is_live == 1` → **Live** (green)
- Service reachable, `conduit_is_live == 0`, not announcing → **Disconnected** (red)

`conduit_idle_seconds` **assistant trigger:** if idle > 12h and connected_clients == 0, the Smart Assistant surfaces a contextual message: "Your station hasn't served any clients recently. New stations can take 24–48 hours to build reputation with the Psiphon network. If your station has been running for several days without traffic, check your network connectivity and public IP."

6.3 Regional Analytics

Top 15 active regions displayed as a table (see Section 5.5 — table rule), aligned with the Regions section of the information architecture.

Column	Source
Flag + country name + ISO code	ISO 3166-1 alpha-2 lookup (in-process, no external API)
Connected clients	`conduit_region_connected_clients{scope,region}` (sum across scopes)
Connecting clients	`conduit_region_connecting_clients{scope,region}` (sum)
Uploaded	`conduit_region_bytes_uploaded{scope,region}` (sum, binary format)
Downloaded	`conduit_region_bytes_downloaded{scope,region}` (sum, binary format)

Rules: regions with zero clients AND zero bytes in all columns are hidden. Scope filter (All / Common / Personal) visible only when personal clients > 0.

6.4 Smart Assistant

Three modes as defined in Section 5.7. Automatic mode deferred to v0.3.0 (see review note — risk on Raspberry Pi restart cascades). v0.2.0 ships Manual and Assisted modes only.

Assisted mode analysis inputs: CPU utilisation (psutil), RAM utilisation, `conduit_connected_clients`, `conduit_bandwidth_limit_bytes_per_second`, `conduit_bytes_uploaded`, `conduit_bytes_downloaded`, `conduit_idle_seconds`.

GUIDE.md baseline: ~150–350 concurrent users per CPU / 2 GB RAM pair. The assistant references this range when computing suggestions.

6.5 Bandwidth Scheduling

Time-based bandwidth profiles using Conduit's built-in reduced-mode mechanism (`InproxyReduced*` via `--set`), presented as the visual timeline defined in Section 5.7.

Fields written to the service unit drop-in: `InproxyReducedStartTime`, `InproxyReducedEndTime`, `InproxyReducedLimitUpstreamBytesPerSecond`, `InproxyReducedLimitDownstreamBytesPerSecond`, `InproxyReducedMaxCommonClients` (optional).

Changes to the schedule follow the same diff + confirm + restart workflow as all other configuration changes.

6.6 Broker Live Status

`conduit_is_live` is the topmost element of the Overview screen and is also present in the Live Clients section. Four states as defined in Section 6.2. Broker disconnected banner displayed per Section 5.8.

6.7 Theme Support

Light / Dark / System themes implemented per Section 5.10. Theme preference persisted via server-side cookie, not localStorage.

7. v0.3.0 — Historical Analytics & Operations

Feature	Description
Historical charts	Time-series charts for connected clients, bytes transferred, bandwidth utilisation. SQLite storage with configurable retention (default 30 days).
Health score	Composite score derived from uptime %, broker live %, average client count, and bandwidth headroom. Displayed on Overview screen.
Smart Assistant — Automatic mode	Adjusts `max_common_clients` within operator-defined bounds. Requires audit log infrastructure from this release.
Backup & restore	Export CCC configuration (not Conduit key) to encrypted archive. Restore from archive.
Update centre	Check for new CCC releases via GitHub releases API. Display release notes. One-click update (runs `update.sh`). No automatic updates — operator must confirm.
Per-direction bandwidth display	Show upstream / downstream limits separately if set in `psiphon_config.json`. Read-only; write deferred.

8. v0.4.0 — Personal Mode & Ryve

Feature	Description
Personal mode setup	Generate compartment ID (`conduit new-compartment-id`), persist to `personal_compartment.json`, display in Config panel.
Pairing token generation	Build v1 pairing tokens (`BuildPersonalPairingToken(id, name)`). Display as QR code and copyable string. Max name: 32 chars.
Personal client limit control	Expose `--max-personal-clients` write path in Config panel (requires compartment ID to be set first). Uses `--max-personal-clients N` flag — not `--set InproxyMaxPersonalClients=N`.

Feature	Description
Scope filter in Regional Analytics	Show common / personal scope breakdown when personal clients > 0.
Ryve claim QR	Invoke `conduit ryve-claim --output <tmp_path>` as subprocess after modal confirmation (see Section 5.8). Serve QR PNG. Delete from disk after display. **Never store or log the private key.**
ProxyID display	Show Curve25519-derived ProxyID from `conduit ryve-claim` output. Display-only.
Ryve rewards	Out of scope until Psiphon exposes rewards data via metrics or a documented API.

Security constraint (Ryve): `conduit ryve-claim` prints the private key to stdout. CCC must capture only the PNG output path and ProxyID field. The private key field must be discarded before any logging or storage. A full-screen modal warning (per Section 5.8) must be shown before invoking the command.

9. Out of Scope

The following items are explicitly out of scope for all planned versions and require formal re-evaluation before being added:

Item	Reason
Ryve rewards / points data	Not exposed in the Conduit binary; requires a documented Ryve API.
Per-user invitation management	Supported in Android app but not in Linux CLI. No CLI mechanism confirmed.
Separate upstream/downstream bandwidth write	Requires changes to `psiphon_config.json`, not the `--set` allowlist. Deferred pending a cleaner abstraction.
Auto-update	Update centre supports one-click update only. No unattended upgrade mechanism.
Multi-node management	CCC manages one Conduit instance per installation. Orchestration of multiple nodes is a separate product concern.
Psiphon config file editing	`psiphon_config.json` is provided externally. CCC does not generate or modify it.

10. Revision History

Version	Date	Author	Notes
1.0	2026-06-10	CCC Development Team	Initial draft. R1 capability audit complete.

Version	Date	Author	Notes
1.1	2026-06-10	CCC Development Team	Added Section 5: D1 — UX/UI Design Milestone as formal gate for v0.2.0. Sections 5–9 renumbered to 6–10. Smart Assistant Automatic mode deferred to v0.3.0. Theme persistence changed from localStorage to server-side cookie.

For questions or contributions, open an issue at the CCC GitHub repository.